02 MAR 2026

# AADK TASK 3

Mini Project: Compose App Feature Combining OOP and Lambdas

MindMatrix

## 1. App Theme – Habit Tracker

For this mini project, I chose the Habit Tracker theme. This feature allows users to track their daily habits like drinking water, exercising, or reading. Each habit can be marked as completed, and users can filter habits based on their completion status.
This theme is useful because it combines object-oriented programming (Habit class) with lambda-based filtering and interactive Compose UI elements.

## 2. Class Structure and Data

### Main Class Design

```
class Habit(
    val name: String,
    val description: String,
    var isCompleted: Boolean,
    val streakCount: Int,
    val category: String
) {
    companion object {
        const val MAX_STREAK = 365
    }
    fun toggleCompletion() {
        isCompleted = !isCompleted
    }
    fun habitStatus(): String {
        return if (isCompleted)
"Completed" else "Pending"
    }
}
```

### Explanation

Properties:

- name → Name of the habit
- description → Short explanation
- isCompleted → Completion status
- streakCount → Number of consecutive days
- category → Type of habit (Health, Study, etc.)

Methods:

- toggleCompletion() → Changes completion state
- habitStatus() → Returns current status

Companion Object:

- MAX_STREAK → Defines maximum possible streak

### List of Habit Objects

```
val habitList = listOf(
    Habit("Drink Water", "Drink 2 liters of water", false, 5, "Health"),
    Habit("Morning Walk", "30 minutes walk", true, 10, "Health"),
    Habit("Read Book", "Read 20 pages daily", false, 3, "Study"),
    Habit("Meditation", "10 minutes meditation", true, 7, "Mindfulness")
)
```

### Higher-Order Function with Lambda

```
fun filterHabits(habits: List<Habit>, condition: (Habit) -> Boolean):
List<Habit> {
    return habits.filter(condition)
}
```

**Example usage:**

```
val completedHabits = filterHabits(habitList) { habit ->
    habit.isCompleted
}
```

Here, the lambda decides the filtering logic. This makes the function reusable.

## 3. Compose UI Integration Plan

### UI Sketch Idea

- Use LazyColumn to display list of habits.
- Each habit is shown inside a Card.
- Show habit name, description, and streak.
- Add a Button to toggle completion.

**Example Compose Snippet**

```
@Composable
fun HabitScreen(habits: List<Habit>) {
    LazyColumn {
        items(habits) { habit ->
            Card {
                Column {
                    Text(habit.name)
                    Text("Streak: ${habit.streakCount}")
                    Text(habit.habitStatus())

                    Button(onClick = { habit.toggleCompletion() }) {
                        Text("Toggle")
                    }
                }
            }
        }
    }
}
```

**UI Response to State Change**

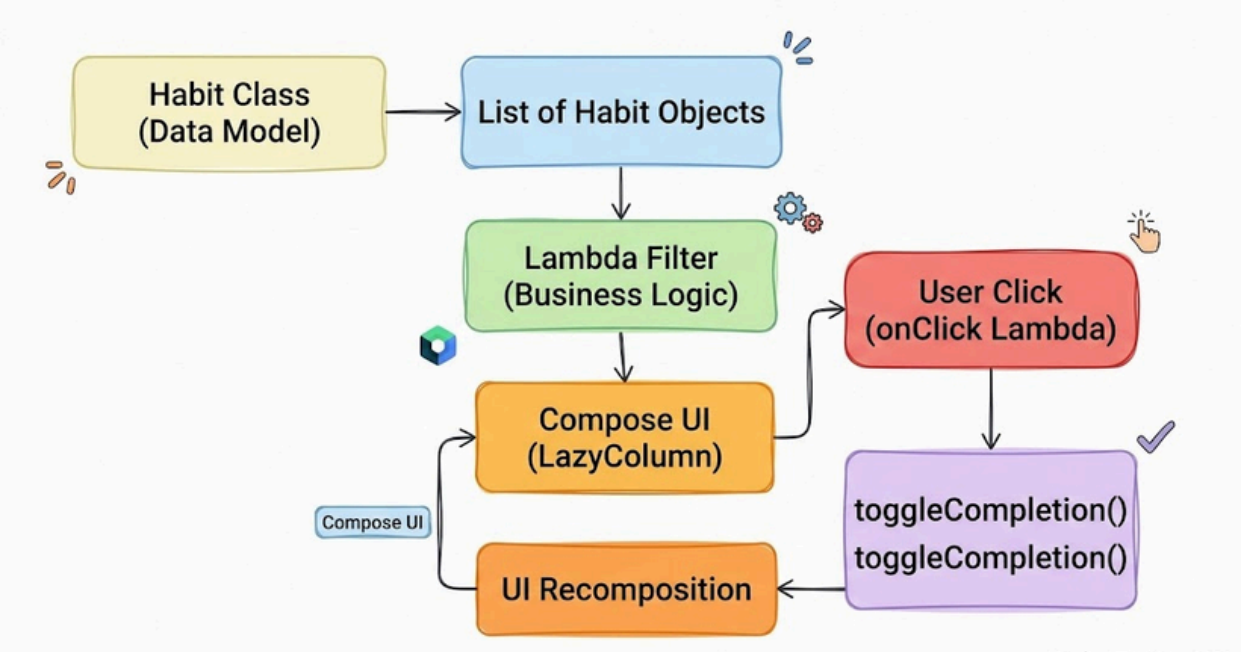
When toggleCompletion() is called inside the onClick lambda:

- isCompleted changes.
- Compose recomposes the UI.
- The displayed status updates automatically.

State handling happens in Compose using remember and mutableStateOf.

## 4. Flow Diagram and Architecture

### Flow Diagram




### Responsibility Breakdown

- Habit Class → Handles business logic
- Lambda filter → Performs data transformation
- Compose UI → Handles state and rendering
- onClick lambda → Connects UI to logic

## 5. Error & Edge Case Planning

### 1. Empty Habit List

- Problem: No habits added.
- Solution: Show a message like "No habits found" instead of empty screen.

### 2. Null Object

- Problem: Habit object might be null (if data comes from API).
- Solution: Use safe call operator ?. before accessing properties.

**Example:**
habit?.name ?: "Unknown Habit"

### 3. User Rapidly Clicking Toggle

- Problem: Multiple quick clicks may cause unexpected behavior.
- Solution: Disable button briefly or manage state properly using Compose state handling.

### 4. Invalid Streak Count

- Problem: Streak exceeding MAX_STREAK.
- Solution: Add validation before updating streak.

**Reflection**

In this task, I combined OOP by designing the Habit class and functional programming by using lambdas for filtering and UI interaction. It helped me understand how both concepts work together in real Android development.

Object-oriented programming helps in organizing data and behavior properly. Functional programming makes logic reusable and flexible. One trade-off I noticed is that mixing too much logic inside UI can reduce clarity, so separation of concerns is important.

If I were to turn this into a real app feature, I would:

- Use ViewModel for proper state management.
- Store habits in a database (Room).
- Add animations and better UI design.
- Add date-based tracking for daily completion.

Overall, this task helped me understand how data models, lambdas, and Compose UI interact together to build a real feature.